

BDH Driven Continuous Narrative Reasoning

Team BDH_ON_TOP

Piyush Panda, Priyam Ghosh, Priyansh Rawat

January 2026

Index

Contents

1	Introduction	3
1.1	Local Setup (Windows / PowerShell)	4
2	Overall Approach	5
2.1	Representation pipeline (from text to decision)	5
2.2	Training objectives and the role of novels	5
2.3	Inference and threshold calibration	6
3	Long-Context Handling	7
3.1	Sequential ingestion with positional context (time/segment embeddings)	7
3.2	CausalMemory recurrent update	7
3.3	Memory trace for downstream use	7
3.4	Operational summary	7
4	Distinguishing Causal Signals from Noise	8
4.1	Phase A: Unsupervised ingestion (stabilizing memory dynamics)	8
4.2	Phase B: Supervised consistency (forcing discriminative alignment)	8
4.3	Why dual-objective training filters noise (in this implementation)	8
4.4	Operational outcome	8
5	Key Limitations and Failure Cases	9

1 Introduction

This document summarizes our overall approach, our long-context strategy, how we distinguish causal signal from noise, and the key limitations and failure cases of the current BDH + structured memory implementation.

Problem framing. We address *narrative consistency detection* in a setting where each sample is a natural-language claim (**content**) paired with an entity identifier (**char**). The objective is to predict whether the claim is consistent with the entity’s accumulated narrative state (**consistent**) or contradicts it (**contradict**). This is challenging because supervision is sparse relative to the space of possible claims, and because the informative evidence within a narrative is typically long-range, distributed, and interleaved with irrelevant narration.

Core design intuition. Instead of relying on a single monolithic sequence model to internally store all prior facts, we externalize persistence into an explicit memory structure. We then treat classification as a similarity decision between (i) a representation of the claim and (ii) a learned *memory trace* for the referenced entity. In this formulation, long context is handled by (a) compressing durable information into memory over time and (b) comparing a claim embedding against a persistent entity-indexed memory trace, limiting dilution from background text.

Pragmatic constraints. Our implementation intentionally prioritizes simplicity and controllability: word-level hashing to pseudo-token ids, mean-pooling for statement embeddings, and gated memory updates that are easy to regularize and debug. These choices let us iterate quickly and observe failure modes clearly (e.g., collisions, negation sensitivity, drift), which in turn informs the long-context and causality heuristics described later in this report.

1.1 Local Setup (Windows / PowerShell)

Expected project layout. Place the code, scripts, and datasets in the following directory structure. The pipeline assumes `train.csv` and `test.csv` reside under `/data`, and that checkpoints are stored under `/checkpoints`.

```
/BDH_ON_TOP_KDSH_2026:
  main.py
  setup.ps1
  requirements.txt
  /checkpoints
    best_bdh_model.pth
  /outputs
    results.csv
  /data
    train.csv
    test.csv
    The Count of Monte Cristo.txt
    In search of the castaways.txt
```

Run instructions. The following sequence installs dependencies via `setup.ps1` and runs the pipeline locally. Because the system uses Python hashing for token/entity indexing, deterministic execution requires controlling Python hash randomization externally via `PYTHONHASHSEED`. We therefore set `PYTHONHASHSEED=0` so that bucket assignments remain stable across runs and saved checkpoints remain semantically compatible.

```
Set-ExecutionPolicy -Scope Process Bypass
Unblock-File .\setup.ps1 -ErrorAction SilentlyContinue

.\setup.ps1 -Compute cpu

$env:PYTHONHASHSEED="0"
python main.py
```

2 Overall Approach

We treat consistency detection as a similarity problem between (i) an embedding of the statement text and (ii) a persistent memory representation queried by the referenced entity identifier. Each sample provides an entity identifier `char` and a textual claim `content`; we predict `consistent` if the claim embedding aligns with the entity’s memory trace, and `contradict` otherwise.

2.1 Representation pipeline (from text to decision)

Text embedding (hashed pseudo-tokenization). We map each whitespace-delimited word to a pseudo-token id using hash-based indexing and embed it with a learned lookup table of dimension D . This acts as a lightweight surrogate for a true tokenizer and vocabulary, trading semantic fidelity for speed and implementation simplicity. We represent a full statement by mean-pooling its word embeddings:

$$\mathbf{b} = \frac{1}{|\mathcal{W}|} \sum_{w \in \mathcal{W}} \text{Enc}(w) \in \mathbb{R}^D.$$

Operationally, $\text{Enc}(w)$ is implemented by an embedding table followed by a layer normalization step. (In our current code, the encoder also contains additional parameters, but the downstream pipeline uses the returned normalized embeddings for pooling.)

Structured memory (persistent state). We maintain a persistent memory matrix $\mathbf{M} \in \mathbb{R}^{N \times D}$ inside a CausalMemory module. This memory serves two roles:

- During **unlabeled ingestion**, memory is updated sequentially using word-indexed slots (bucketed by `hash(word)`).
- During **supervised consistency**, memory is queried using entity-indexed slots (bucketed by `hash(char)`).

Memory is updated with a gated write/retain rule. Given the current slot vector $\mathbf{m}$ and a contextualized fact vector $\tilde{\mathbf{f}}$, we compute an input gate $i \in (0, 1)$ and a forget gate $f \in (0, 1)$, then update:

$$\mathbf{m}' = f \cdot \mathbf{m} + i \cdot \tilde{\mathbf{f}}.$$

We apply a reconstruction head $\text{Rec}(\cdot)$ to map memory back into the embedding space, producing a memory trace $\mathbf{r} = \text{Rec}(\mathbf{m}) \in \mathbb{R}^D$ comparable to statement embeddings.

Score function and prediction. For a labeled pair $(\text{char}, \text{content})$, we compute statement embedding $\mathbf{b}$ and entity trace $\mathbf{r} = \text{Rec}(\mathbf{M}[e])$ with $e = \text{hash}(\text{char}) \bmod N$, then score with cosine similarity:

$$s = \cos(\mathbf{r}, \mathbf{b}).$$

A higher score indicates alignment (consistent), while a lower score indicates mismatch (contradict). This similarity view makes the decision layer interpretable: failures typically correspond to either a weak $\mathbf{b}$ representation (bag-of-words limitations) or degraded memory traces due to collisions, drift, or entity/ingestion mismatch.

2.2 Training objectives and the role of novels

Unlabeled corpus ingestion (novel-driven memory dynamics). To stabilize memory dynamics and train the gating/reconstruction behavior under long streams, we ingest text from novels sequentially. We iterate through chapters and words, embed each word into $\mathbf{f}_t$, modulate it with lightweight positional context (a time index within the processed segment and a segment/chapter index), update the corresponding *word-indexed* memory slot, and train a reconstruction objective so that $\text{Rec}(\mathbf{m}')$ recovers the observed embedding. Concretely, we optimize:

$$\mathcal{L}_{\text{recon}} = \|\text{Rec}(\mathbf{m}') - \mathbf{f}_t\|_2^2,$$

with auxiliary gate regularizers that encourage conservative writes (small average i) and stable retention (large average f), reducing catastrophic overwrites under long text.

Supervised consistency learning (margin-style alignment). Using labeled pairs from `train.csv`, we align each entity’s memory trace with consistent statements and push it away from contradictory ones. Given score $s = \cos(\mathbf{r}, \mathbf{b})$, we apply a hinge-style margin objective in similarity space that enforces high similarity for `consistent` and low similarity for `contradict`. Intuitively, consistent samples should yield high cosine similarity to the entity slot; contradictory samples should be separated by a margin.

Why this hybrid training helps (and what it does *not* guarantee). The novel ingestion phase regularizes the *mechanics* of memory (gates, reconstruction stability, resistance to drift). The supervised phase provides the *task signal* that links entity slots to label-discriminative statements. Importantly, because ingestion currently updates `hash(word)` slots while supervised inference queries `hash(char)` slots, ingestion does *not* directly pre-fill entity slots unless the addressing scheme is modified or entity identifiers are explicitly ingested. In the current codebase, ingestion primarily improves stability/conditioning rather than guaranteeing direct entity-memory initialization.

2.3 Inference and threshold calibration

At prediction time, we calibrate a similarity threshold τ on training data by sweeping candidate values and selecting the one that maximizes balanced accuracy. For each test sample, we compute $s = \cos(\mathbf{r}, \mathbf{b})$ and predict:

$$\hat{y} = \begin{cases} \text{consistent}, & s \geq \tau \\ \text{contradict}, & s < \tau. \end{cases}$$

This explicit thresholding is simple and stable to deploy, but it remains sensitive to distribution shift.

Reproducibility note (hash stability). Because the system uses hashing for token/entity indexing, deterministic execution requires controlling Python hash randomization. We therefore set `PYTHONHASHSEED=0` in the local setup steps so that bucket assignments remain stable across runs and saved checkpoints remain semantically compatible.

3 Long-Context Handling

Motivation. For long narratives, a single pooled embedding of `content` is a lossy bottleneck: salient facts can be diluted, event ordering is collapsed, and late-arriving narration can dominate. Our implementation therefore handles long context by treating the narrative as a *sequential stream* and compressing it online into a persistent memory matrix via recurrent gated updates. In the current codebase, long-context is handled during ingestion by sequentially updating memory; the supervised consistency pathway continues to represent each claim by a mean-pooled statement embedding.

3.1 Sequential ingestion with positional context (time/segment embeddings)

As the model ingests a novel word-by-word, each word is embedded:

$$\mathbf{f}_t = \text{Enc}(w_t) \in \mathbb{R}^D.$$

To reduce destructive interference across long streams, we inject two learned positional-context projections:

- **Time embedding** (via `time_proj`): uses the word index t within the processed segment.
- **Segment embedding** (via `loc_proj`): uses the segment/chapter index l in the processed subset.

Let $\mathbf{t}_t = \tanh(\text{time_proj}(t))$ and $\ell_l = \tanh(\text{loc_proj}(l))$. We contextualize:

$$\tilde{\mathbf{f}}_t = \mathbf{f}_t \odot (\mathbf{t}_t + \ell_l).$$

This makes repeated tokens behave differently depending on when and in which segment they occur, improving stability of recurrent updates.

3.2 CausalMemory recurrent update

Memory is a matrix $\mathbf{M} \in \mathbb{R}^{N \times D}$. At each ingestion step, we select a slot by hashing the current word:

$$e_t = \text{hash}(w_t) \bmod N.$$

Given the current slot vector $\mathbf{m}_t = \mathbf{M}[e_t]$, the CausalMemory module computes gates:

$$i_t = \sigma(g_i([\tilde{\mathbf{f}}_t; \mathbf{m}_t])), \quad f_t = \sigma(g_f([\tilde{\mathbf{f}}_t; \mathbf{m}_t])),$$

and performs the recurrent update:

$$\mathbf{m}_{t+1} = f_t \cdot \mathbf{m}_t + i_t \cdot \tilde{\mathbf{f}}_t.$$

In our implementation, the memory slot is then written back through an explicit state assignment, making $\mathbf{M}$ behave as a persistent external buffer updated by learned gates.

3.3 Memory trace for downstream use

We map memory back into the embedding space using a reconstruction head:

$$\mathbf{r} = \text{Rec}(\mathbf{m}) \in \mathbb{R}^D.$$

During supervised consistency, we query an entity-indexed slot $e = \text{hash}(\text{char}) \bmod N$ and compare $\mathbf{r} = \text{Rec}(\mathbf{M}[e])$ to the statement embedding $\mathbf{b}$. The current codebase does not implement chunk-level retrieval pooling at inference; long-context compression occurs through sequential memory updates during ingestion.

3.4 Operational summary

We handle long context by treating the narrative as a sequential stream. As the model ingests the novel word-by-word, it updates the persistent memory matrix $\mathbf{M}$ using learned time and segment embeddings (`time_proj` and `loc_proj`) to reduce destructive overwrites. In the current implementation, inference remains a similarity comparison between an entity-indexed memory trace and a mean-pooled statement embedding.

4 Distinguishing Causal Signals from Noise

Objective. In long narratives, much text is incidental (stylistic narration, descriptive filler, repeated phrasing). Our codebase does not perform explicit intervention/ablation tests at inference. Instead, we distinguish signal from noise through *dual-objective training*: an unsupervised ingestion objective that stabilizes recurrent memory dynamics over long streams, and a supervised consistency objective that forces entity-indexed memory traces to align with discriminative facts needed for classification.

4.1 Phase A: Unsupervised ingestion (stabilizing memory dynamics)

During ingestion, we update memory sequentially (Section 3) and optimize a reconstruction objective:

$$\mathcal{L}_{\text{recon}} = \|\text{Rec}(\mathbf{m}_{t+1}) - \mathbf{f}_t\|_2^2.$$

To discourage indiscriminate writing and reduce drift, we regularize the gates:

$$\mathcal{L}_{\text{gate}} = \lambda_i \mathbb{E}[|i_t|] + \lambda_f \mathbb{E}[|1 - f_t|].$$

Operationally, this biases the system toward conservative updates (small i_t on average) and strong retention (large f_t on average), improving stability under long streams.

4.2 Phase B: Supervised consistency (forcing discriminative alignment)

For labeled pairs $(\text{char}, \text{content}, y)$, we compute a statement embedding $\mathbf{b}$ and retrieve an entity trace $\mathbf{r} = \text{Rec}(\mathbf{M}[e])$ with $e = \text{hash}(\text{char}) \bmod N$, then score:

$$s = \cos(\mathbf{r}, \mathbf{b}).$$

We train with a hinge-style margin loss that pushes similarity up for **consistent** samples and down for **contradict** samples. In code, this is implemented by enforcing a positive margin for consistent examples and a negative margin for contradictory ones using ReLU clamps.

4.3 Why dual-objective training filters noise (in this implementation)

The combined objective

$$\mathcal{L} = \mathcal{L}_{\text{recon}} + \mathcal{L}_{\text{gate}} + \beta \mathcal{L}_{\text{sup}}$$

implicitly emphasizes useful evidence:

- **Ingestion** trains the memory update mechanism to be stable and context-sensitive (time/segment conditioning).
- **Supervision** only reinforces memory representations that improve label prediction; background narration that does not support discrimination provides weak or inconsistent gradients.
- **Gate regularization** further suppresses writing high-frequency but non-discriminative signals into memory.

This is an implicit filtering mechanism arising from optimization, rather than explicit test-time attribution.

4.4 Operational outcome

We distinguish signal from noise through our hybrid loss function. The ingestion phase stabilizes the causal memory dynamics under long text streams (unsupervised), while the supervised consistency phase forces the entity-indexed memory trace to align with discriminative facts needed for classification. No explicit intervention or ablation scoring is performed at inference in the current codebase.

5 Key Limitations and Failure Cases

Hash-based indexing and reproducibility. We map words and entities to buckets via hashing, $\text{id}(x) = \text{hash}(x) \bmod K$. Python’s hash is randomized across sessions unless `PYTHONHASHSEED` is fixed, so index assignments can change between runs, breaking checkpoint semantic compatibility. Hashing also induces collisions, causing interference between unrelated tokens/entities.

Bag-of-words embedding collapses syntax and scope. Mean pooling discards word order, negation scope, and compositional semantics. This yields failures on negations, quantifiers, temporal qualifiers, and role reversals (e.g., “*X killed Y*” vs “*Y killed X*”).

Entity-memory and corpus-ingestion mismatch. Corpus ingestion updates memory via `hash(word)` while supervised consistency queries memory via `hash(char)`. In the current implementation, ingestion therefore trains memory dynamics and reconstruction behavior but does not guarantee that entity-indexed slots are directly populated by relevant narrative facts.

Memory slot collisions and cross-entity contamination. Distinct entities can map to the same memory slot, leading to overwriting and attribute leakage. This is especially problematic when the number of unique entities is large relative to N .

Catastrophic drift under long or repetitive context. Repeated writes can cause memory drift toward recent narrative rather than stable facts, reducing discriminability and producing decision flips when late-arriving contradictory text dominates. Gate regularization mitigates but does not eliminate this.

Spurious similarity and lexical overlap bias. Cosine similarity over averaged embeddings is sensitive to shared vocabulary. Contradictions with high lexical overlap (e.g., *alive* vs *dead*) can be misclassified as consistent, while true paraphrases with low overlap can be missed.

Sensitivity to tokenization artifacts. Whitespace splitting and hashing are brittle under punctuation, casing, possessives, and OCR noise. Variants such as *John*, *John,*, and *john* behave as different tokens, increasing sparsity and collision noise.

Weak handling of co-reference and implicit entities. Without co-reference resolution, evidence expressed via pronouns or titles may not align with the correct entity memory trace, degrading performance when the entity is not explicitly named.

Ambiguity and multi-entity statements. Relational statements (e.g., “*X betrayed Y*”) are compared against a single entity trace `r(char)`. Without explicit relation modeling, the system can misattribute roles and conflate evidence.

Threshold calibration instability. A single global threshold τ can be brittle under distribution shift (different entity frequency, different writing style, different content length). Rare entities are particularly sensitive due to under-trained entity-indexed slots.

Optimization and state-update hazards. In the current codebase, memory state is updated both through learned gates and through explicit state assignment (writing back the updated slot). This hybrid mechanism can create non-stationary dynamics and makes convergence sensitive to learning rate and loss weights.

Compute and batching limitations. Per-word loops and frequent optimizer steps underutilize the GPU and encourage truncation of ingestion (limited chapters/words), biasing what the memory sees and reducing coverage of relevant narrative patterns.